



Amazon S3 Storage Performance

S3 Transfer Acceleration – Overview

Amazon S3 Transfer Acceleration uses Amazon CloudFront's globally distributed edge locations to speed up uploads and downloads between clients and a specific S3 bucket. It's beneficial for applications serving global users.

How to Enable

- Users frequently access or upload objects from geographically distant locations.
- Latency-sensitive use cases (e.g., media uploads, backups, or content sharing).
- Frequent uploads of large files to S3.

When to Enable

Can be enabled via:

- **AWS Console** → S3 → [Bucket] → Properties
→ Transfer Acceleration → Enable

Performance Benefits

- Faster transfer speeds by reducing network hops.
- Utilises optimised network routes via CloudFront.

PROS AND CONS

- **Pros**

- Simple to enable.
- Can significantly improve data transfer time from distant locations.

- **Cons**

- Incurs additional costs.
- Not effective for all scenarios (e.g., same-region clients see little or no improvement).
- Cannot be used with all S3 features (e.g., some older SDKs may not support it).

Analyse Activity Metrics

- **Metric**
- **Sample Value (per day)**
- **Interpretation**
- **GetRequests**
- 250,000
- High read activity; object download is frequent.
- **PutRequests**
- 35,000
- Moderate write activity; fewer uploads than downloads.
- **BytesDownloaded**
- 500 GB
- Heavy outbound data transfer.
- **BytesUploaded**
- 100 GB
- Less frequent uploads.

Key Findings

- The system experiences **5x more downloads than uploads**.
- Most operations are **GETs**, which may strain performance under high concurrency.
- No severe write bottlenecks, but **download patterns** need tuning for cost and latency.

Potential Bottlenecks

- Download traffic could saturate bandwidth.
- Large objects or non-compressed assets may inflate transfer times.
- Insufficient edge caching or regional replication might be causing user latency.

Tools Used

- Amazon CloudWatch Metrics
- S3 Storage Class Analysis
- AWS Console metrics dashboard

Uploading

13%

Cancel

Total remaining: 214 files: 4.4 MB (87.16%)

Estimated time remaining: 3 minutes

Transfer rate: 21.9 KB/s

Summary

Destination

s3://code-bucket9090

Succeeded

45 files, 667.5 KB (12.84%)

Failed

0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (259 total, 5.1 MB)

Find by name

< 1 2 3 4 5 6 7 ... 26 >

Name	Folder	Type	Size	Status	Error
.DS_Store	tenio/	-	8.0 KB	✓ Succeeded	-
.graphqlconfig	tenio/tenio-insurance-backend/	-	212.0 B	✓ Succeeded	-
.DS_Store	tenio/tenio-insurance-backend/	-	8.0 KB	✓ Succeeded	-
Dockerfile.alpine	tenio/tenio-insurance-backend/	-	469.0 B	✓ Succeeded	-
LICENSE	tenio/tenio-insurance-backend/	-	1.1 KB	✓ Succeeded	-
Dockerfile	tenio/tenio-insurance-backend/	-	469.0 B	✓ Succeeded	-
nest-cli.json	tenio/tenio-insurance-backend/	application/json	285.0 B	✓ Succeeded	-
run.sh	tenio/tenio-insurance-backend/	text/x-sh	85.0 B	✓ Succeeded	-
README.md	tenio/tenio-insurance-backend/	text/markdown	14.6 KB	✓ Succeeded	-
docker-compose.db.yml	tenio/tenio-insurance-backend/	application/x-yaml	268.0 B	✓ Succeeded	-

Troubleshooting Errors

403 Forbidden

- IAM roles or bucket policies may be too restrictive.
- Signed URLs may be expired or invalid.
- Recommendation: Review IAM roles and bucket policies using AWS IAM Access Analyser.

404 Not Found

- Objects have been moved, deleted, or have incorrect paths.
- Recommendation: Implement versioning and lifecycle logging. Validate object paths in code.

Status Code Metrics

Status Code Metric	Value (per day)	Implications
200OKStatusCount	280,000	Most requests succeed.
403ForbiddenErrorCount	500	Potential permission issues or control errors.
404NotFoundErrorCount	1,200	Users or applications request nonexistent objects.

Recommendations for Improving Performance

- **1. Enable Transfer Acceleration for Global Users**
 - Benefit: Reduces latency for distant users.
 - Action: Enable via AWS Console or CLI for buckets with global traffic.
- **2. Introduce CloudFront for Frequent Downloads**
 - Benefit: Reduces 403/404 errors and download latency.
 - Action: Deploy CloudFront in front of S3 with caching and custom error responses.
- **3. Monitor & Optimise Object Sizes**
 - Benefit: Large downloads take time and cost more.
 - Action: Compress large files before upload. Use multipart uploads for large files.

Bonus Practices

- **Tag frequently accessed objects** and move infrequent ones to cheaper storage classes.
- **Implement retry logic** for applications to handle 403/404 gracefully.
- **Set up S3 Access Logs** to audit and correlate failed requests with sources.

Conclusion

Through analysis of usage patterns and error metrics, it is evident that the S3 workload is read-heavy and supports a global user base. Leveraging tools like S3 transfer acceleration, CloudFront, and CloudWatch can significantly enhance performance, reduce errors, and improve user experience. By applying the outlined recommendations, there would be reduced latency, improved reliability, and better operational visibility.

Name: Richmond Kobby Nyarko